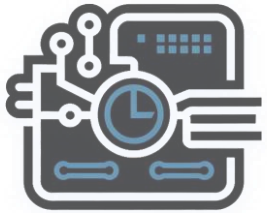


Progettazione Elettronica Digitale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

L02 - Sommario

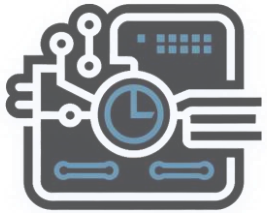
- ➔ Forme canoniche
- ➔ NAND/NOR come porte universali
- ➔ Mappe di Karnaugh
- ➔ Cenni su tecniche automatiche
- ➔ Risposta nel tempo e minimizzazione



Forme Canoniche

- Cosa sono le forme canoniche?
- Mintermini e maxtermini
- Rappresentazione a indice di mintermini e maxtermini
- Rappresentazioni a somme di mintermini (SOM)
- Rappresentazioni a prodotti di maxtermini (POM)
- Rappresentazioni dei complementi delle funzioni
- Conversioni tra rappresentazioni

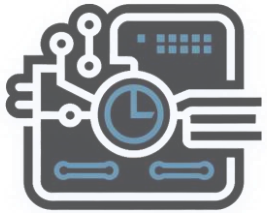
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Forme Canoniche

- È utile specificare le funzioni Booleane in una forma, detta **canonica**, che:
  - permetta di determinare se sono identiche
  - sia in corrispondenza con le tabelle di verità
- Forme canoniche di uso comune:
  - somma di **mintermini** (SOM)
  - prodotto di **maxtermini** (POM)
  - queste forme canoniche portano a **rappresentazioni a due livelli**

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

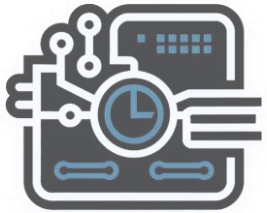


Mintermini

- I **mintermini**, o prodotti fondamentali, sono termini AND in cui ogni variabile è presente in forma affermata o negata
- Una funzione di *n* variabili ha 2^n mintermini
- Esiste una sola combinazione di variabili per cui un certo mintermine valga 1

$$f(X_1, X_2, X_3) = X_1 \overline{X_2} X_3$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

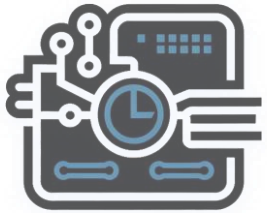


Maxtermini

- I **maxtermini**, o somme fondamentali, sono termini OR in cui ogni variabile è presente in forma affermata o negata
- Una funzione di *n* variabili ha 2^n maxtermini
- Esiste una sola combinazione di variabili per cui un certo maxtermine valga 0

$$f(X_1, X_2, X_3) = X_1 + \overline{X_2} + X_3$$

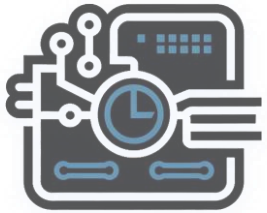
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Ordine standard

- Mintermini e maxtermini sono identificati da un pedice
- Il pedice è un numero che corrisponde a una configurazione binaria
- I bit nella configurazione rappresentano lo stato affermato o negato di ogni variabile secondo un ordine standard
- Tutte le variabili compariranno in un mintermine o maxtermine secondo lo stesso ordine (in genere quello alfabetico)
- Per i mintermini:
  - "*1*" significa che la variabile appare in forma affermata
  - "*0*" significa che la variabile appare in forma negata
- Per i maxtermini:
  - "*0*" significa che la variabile appare in forma affermata
  - "*1*" significa che la variabile appare in forma negata

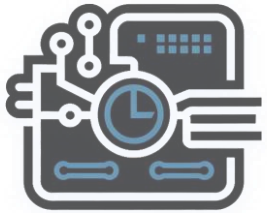
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio di indice con 3 variabili

- Le variabili sono X, Y e Z
- L'ordine standard è X, poi Y, infine Z
- Consideriamo l'indice $0_{10} = 000_2$ per l'esempio a 3 variabili
- Tutte e 3 sono in forma negata per il mintermine 0 (m_0) e nessuna variabile è in forma negata per il maxtermine 0 (M_0)
  - Il mintermine m_0 è $(\bar{X}\bar{Y}\bar{Z})$.
  - Il maxtermine M_0 è $(X + Y + Z)$.

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Relazione tra mintermini e maxtermini

- Dal teorema di De Morgan

$$\overline{x \cdot y} = \bar{x} + \bar{y} \text{ e } \overline{x + y} = \bar{x} \cdot \bar{y}$$

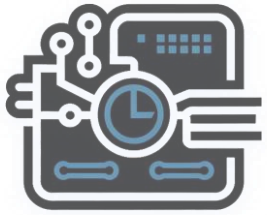
- Esempio con 2 variabili

$$m_2 = x \cdot \bar{y} \text{ e } \mathrm{M}_2 = \bar{x} + y$$

- Si ha che M_2 è il *complemento* di m_2 e viceversa
  - $M_2 = \overline{m_2}$ e $m_2 = \overline{M_2}$

- L'affermazione è generalizzabile al caso di *n* variabili

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Mintermini, maxtermini e tabelle di verità

A1

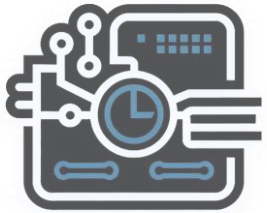
- Possiamo realizzare qualsiasi funzione mettendo in “OR” i mintermini che corrispondono a “1” nella tabella della funzione. Questi sono chiamati i mintermini della funzione

- Possiamo realizzare qualsiasi funzione mettendo in “AND” i maxtermini che corrispondono a “0” nella tabella della funzione. Questi sono chiamati i maxtermini della funzione

- Questo produce due forme canoniche in grado di esprimere qualsiasi funzione Booleana:
  - somma di mintermini (SOM)
  - prodotto di maxtermini (POM)

- Si deriva tale risultato a partire dal teorema di espansione di Shannon
  - Argomento di approfondimento individuale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

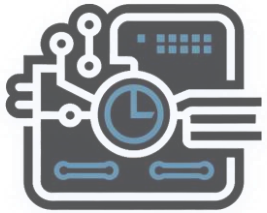


Somma canonica di mintermini

- Tutte le funzioni Booleane possono essere espresse come somme di mintermini
- A partire dalla tabella della funzione, i mintermini usati sono i termini che corrispondono agli 1
- A partire da una espressione generica, espandere tutti i termini per elencare esplicitamente tutti i mintermini. Per ogni termine in cui manca una generica variabile x, mettere in AND con un termine nella forma ($x+\bar{x}$)

$$
\begin{aligned}
f(x,y) &= x + \bar{x}\bar{y} \\
f &= x \cdot (y + \bar{y}) + \bar{x}\bar{y} \\
f &= xy + x\bar{y} + \bar{x}\bar{y} \\
f &= m_3 + m_2 + m_0 = \sum_m (3,2,0)
\end{aligned}
$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

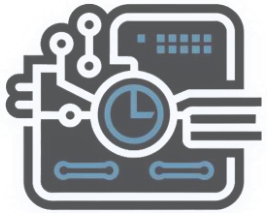


Prodotto canonico di maxtermini

- Tutte le funzioni Booleane possono essere espresse come prodotti di maxtermini
- A partire dalla tabella della funzione, i maxtermini usati sono i termini che corrispondono agli 0
- A partire da una espressione generica, espandere tutti i termini per elencare esplicitamente tutti i maxtermini. Applicare poi la seconda proprietà distributiva, mettendo in OR per ogni termine in cui manca una generica variabile x, un termine nella forma $(x \cdot \bar{x})$. Applicare nuovamente la proprietà distributiva

$$f(x,y,z) = x + \bar{x}\bar{y}$$
$$f = (x + \bar{x}) \cdot (x + \bar{y}) = x + \bar{y}$$
$$f = x + \bar{y} + z\bar{z}$$
$$f = (x + \bar{y} + z) \cdot (x + \bar{y} + \bar{z})$$
$$f = M_2 \cdot M_3 = \prod_M (2,3)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Complemento di una funzione

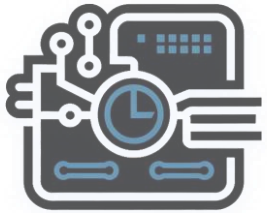
- Il **complemento** di una funzione espressa come somma di mintermini si ottiene selezionando i mintermini mancanti nella forma canonica somma di mintermini
- In alternativa, il complemento di una funzione espressa come somma di mintermini è semplicemente il prodotto dei maxtermini con gli stessi indici

$$f(x,y,z) = \sum_m (1,3,5,7)$$

$$\overline{f}(x,y,z) = \sum_m (0,2,4,6)$$

$$\overline{f}(x,y,z) = \prod_M (1,3,5,7)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Conversione tra forme

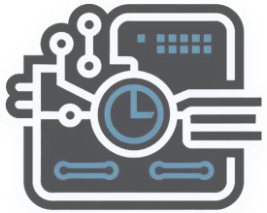
- Per convertire tra forme SOM e POM è sufficiente procedere come segue:
  - Trovare il complemento della funzione scambiando i termini nella lista con quelli che non vi appaiono
  - Cambiare i prodotti in somme o viceversa

$$f(x,y,z) = \sum_m (1,3,5,7)$$

$$\overline{f}(x,y,z) = \sum_m (0,2,4,6)$$

$$f(x,y,z) = \prod_M (0,2,4,6)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Forme standard

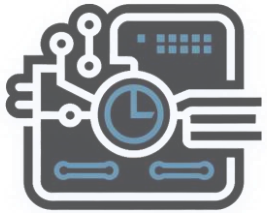
- Forma standard *Somma di Prodotti* (SOP)
  - Le espressioni sono scritte come OR di termini AND
- Forma standard *Prodotti di Somme* (POS)
  - Le espressioni sono scritte come AND di termini OR
- Esempi di forme standard:

$$f_{SOP} = xyz + \overline{z}x + y$$
$$f_{POS} = (x + y + z) \cdot (\overline{z} + x) \cdot \overline{z}$$

- Forme miste non standard:

$$f = (xy + z) \cdot (x + z)$$
$$f = xy\overline{z} + xy \cdot (x + z)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

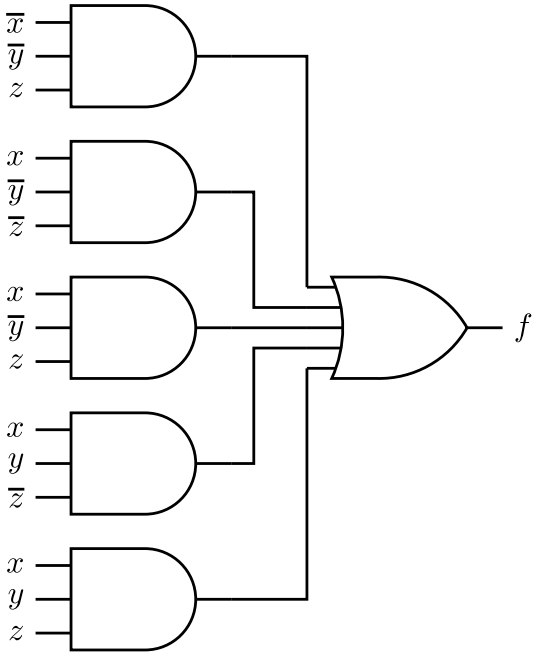


Somma-di-Prodotti (SOP) standard

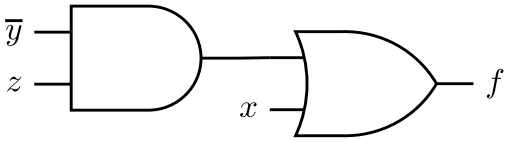
- Una forma a somma di mintermini per *n* variabili può essere derivata direttamente da una tabella di verità
- L'implementazione di questa forma è una rete a due livelli di porte logiche tale che:
  - Il 1° livello consiste di porte AND a *n*-ingressi e
  - Il 2° livello è una singola porta OR gate (con meno di 2^n ingressi)
- Questa forma può essere minimizzata in modo da rendere più semplice il circuito

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Forma standard SOP

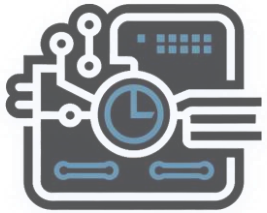


$$
\begin{aligned}
f(x,y,z) &= \sum_m (1,4,5,6,7) \\
&= \bar{x}\bar{y}z + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z} + xyz \\
&= \bar{x}\bar{y}z + x(\bar{y}\bar{z} + \bar{y}z + y\bar{z} + yz) \\
&= \bar{x}\bar{y}z + x(y+\bar{y})(z+\bar{z}) \\
&= \bar{x}\bar{y}z + x \\
&= \bar{y}z + x
\end{aligned}
$$



- La funzione semplificata contiene 3 letterali rispetto ai 15 della versione a mintermini
- Le due funzioni, seppur equivalenti, hanno anche rappresentazioni circuitali dal costo significativamente diverso

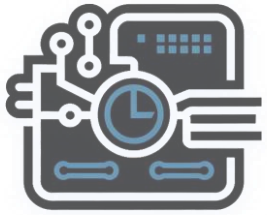
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Considerazioni su SOP e POS

- Gli esempi precedenti mostrano che:
  - Le forme canoniche (somma-di-mintermini, prodotto-di-maxtermini), o altre forme standard (SOP, POS) hanno complessità diversa
  - L'algebra Booleana può essere usata per manipolare le espressioni e semplificarle
  - Espressioni più semplici portano a implementazioni a 2 livelli più semplici
- Domande aperte:
  - Come ottenere l'espressione più "semplice"?
  - Il circuito a costo minimo è unico?

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



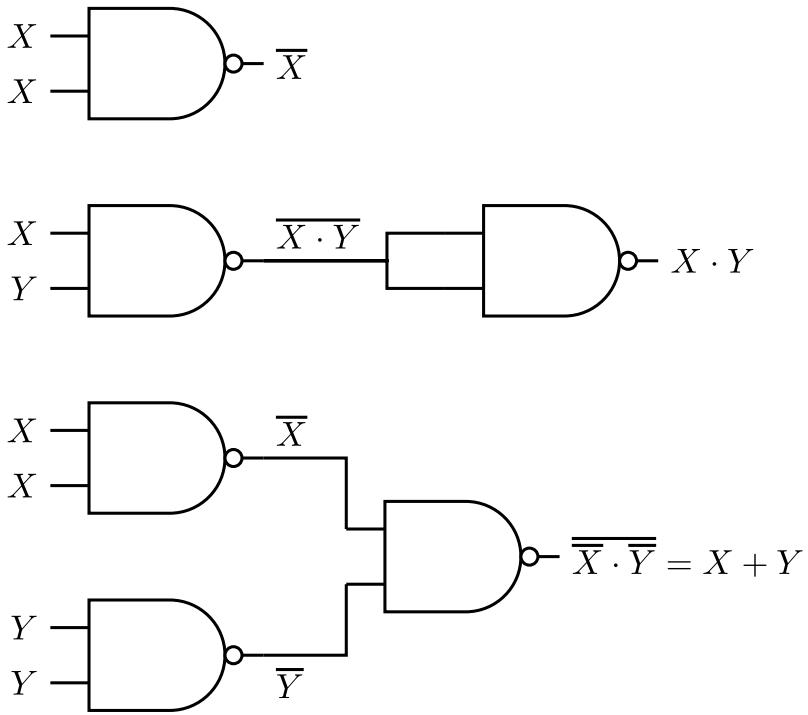
Insiemi di operazioni funzionalmente completi

- L’algebra Booleana si basa su un insieme di due elementi {0,1} e su un insieme di operatori {AND, OR, NOT}
- Tramite il teorema di De Morgan è possibile derivare AND e OR l’uno dall’altro, per cui è sufficiente uno dei due, insieme al NOT per costruire l’intera algebra Booleana
- Ci sono due ulteriori insiemi di particolare rilevanza, in quanto funzionalmente complete, da cui è possibile derivare tutta l’algebra Booleana
  - Insieme {NAND}
  - Insieme {NOR}
- L’attenzione posta sulle porte NAND e NOR è legata alla convenienza di implementazione, per area e velocità, nella tecnologia CMOS
  - I due operatori però non hanno un simbolo dedicato poiché mancano entrambi della proprietà associativa

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Insieme {NAND}

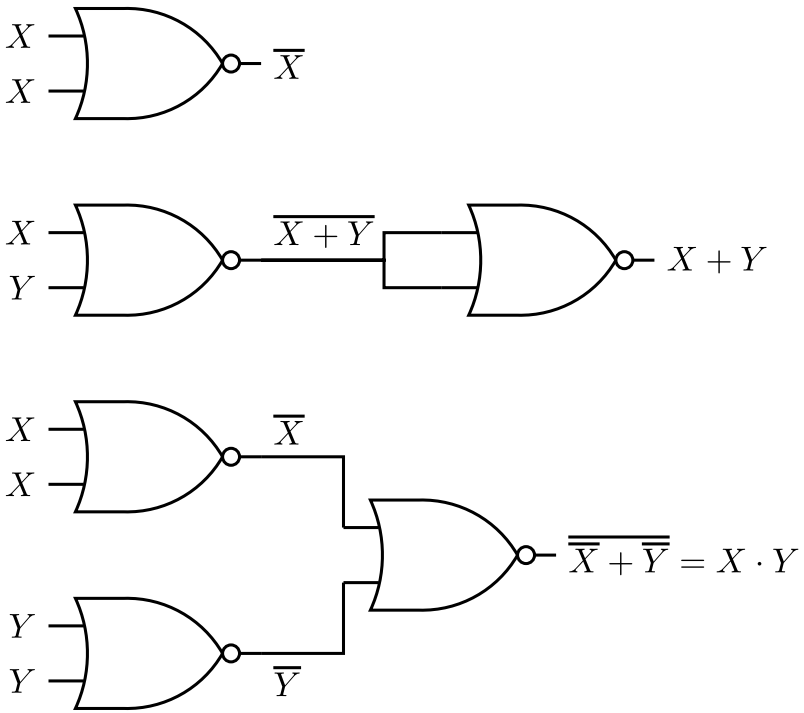
- È possibile ottenere gli operatori NOT, AND, e come conseguenza di De Morgan, l'operatore OR, combinando porte NAND come rappresentato a seguire



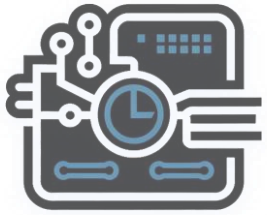
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Insieme {NOR}

- È possibile ottenere gli operatori NOT, OR, e come conseguenza di De Morgan, l'operatore AND, combinando porte NOR come rappresentato a seguire



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Semplificazione a 2 livelli

- Si fonda sul **teorema di unificazione**: $x(y + \overline{y}) = x$

$$f = x\overline{y} + xy = x(y + \overline{y}) = x$$

x	y	f
0	0	0
0	1	0
1	0	1
1	1	1

$$f = \bar{x}\bar{y} + x\bar{y} = (x + \bar{x})\bar{y} = \bar{y}$$

x	y	f
0	0	1
0	1	0
1	0	1
1	1	0

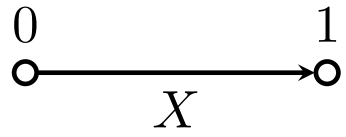
- Lo scopo è identificare sottoinsiemi di due elementi nell'*on-set* in cui solo una variabile cambia. Tale variabile può essere eliminata
- Definiamo on-set (off-set) l'insieme delle combinazioni di input tali per cui la funzione assume il valore 1 (0)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

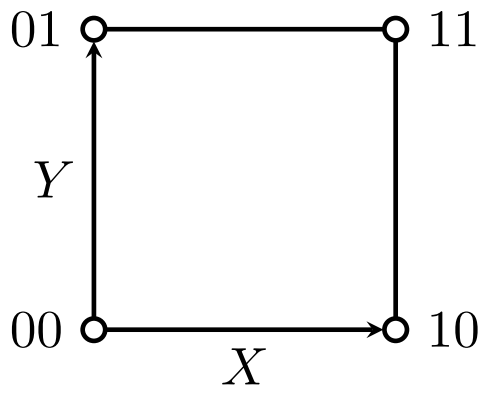
Cubi Booleani

A2

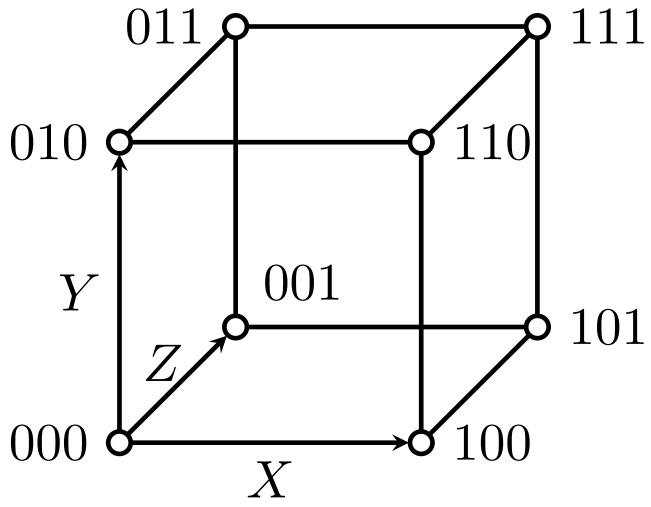
- Sono una tecnica visuale che permette di identificare più facilmente l'applicabilità del teorema di unificazione
- Un cubo ha tante dimensioni quante sono le variabili considerate
- Argomento di approfondimento individuale



1−cube



2−cube

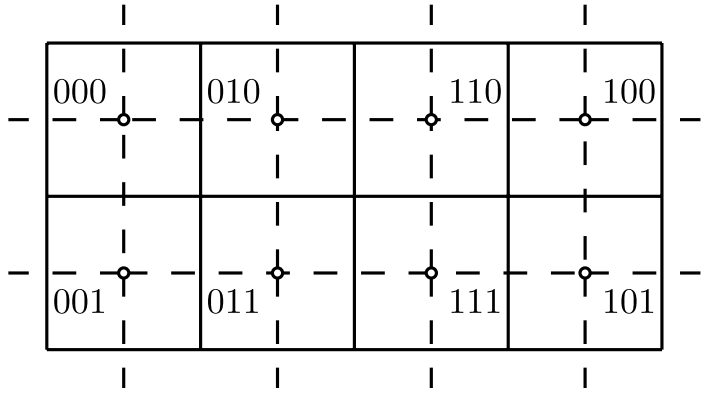


3−cube

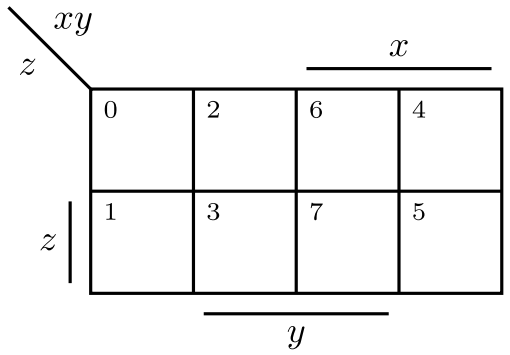
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

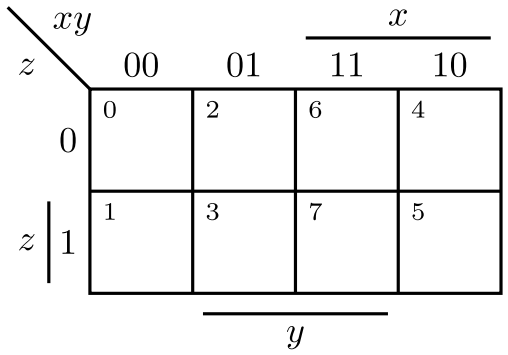
Dai cubi alle mappe di Karnaugh

- Trasformiamo il cubo tridimensionale in una rappresentazione planare cercando di rispettare la disposizione e l'adiacenza dei vertici nello spazio di partenza, per costruire la **mappa di Karnaugh** corrispondente

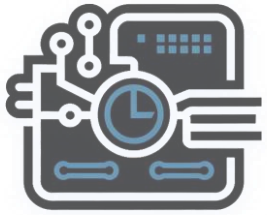


z \ xy	00	01	11	10
0	0	2	6	4
1	1	3	7	5





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Mappe di Karnaugh (K-map)

- Una **K-map** è una collezione di caselle:
  - Ogni casella rappresenta un mintermine (o un maxtermine)
  - La collezione di caselle è una rappresentazione grafica di una funzione Booleana
  - Caselle adiacenti differiscono per il valore di una variabile
  - Si possono identificare diverse espressioni algebriche della stessa funzione riconoscendo configurazioni di caselle

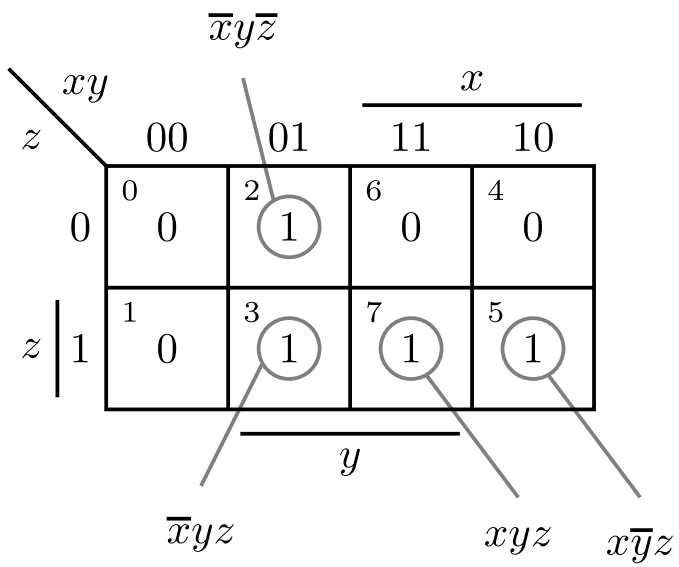
- La K-map può essere vista come:
  - Una versione riorganizzata della tabella di verità

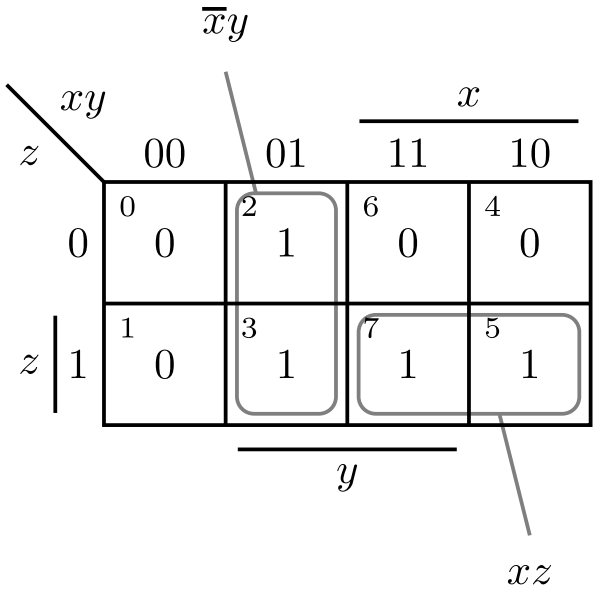
- Sono un mezzo per:
  - Trovare forme standard ottime o quasi-ottime, come SOP, POS, circuiti a due livelli (per funzioni con poche variabili) e per visualizzare e manipolare espressioni Booleane

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio: K-map di funzione in 3 variabili

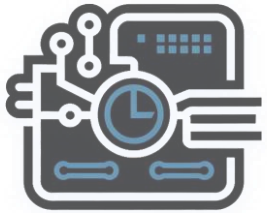
x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1





$$f(x,y,z) = \sum_m (2,3,5,7) = \overline{x}y\overline{z} + \overline{x}yz + x\overline{y}z + xyz = \overline{x}y + xz$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

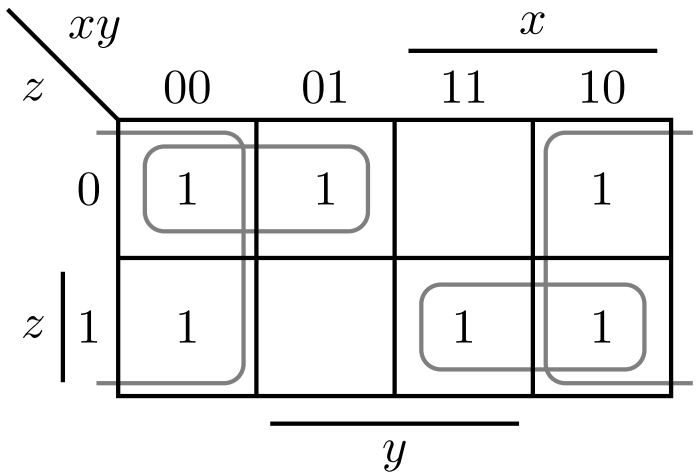


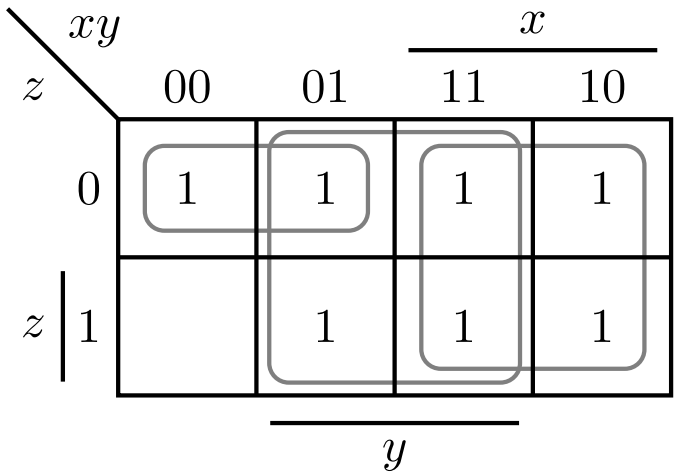
Combinazione di caselle su K-map

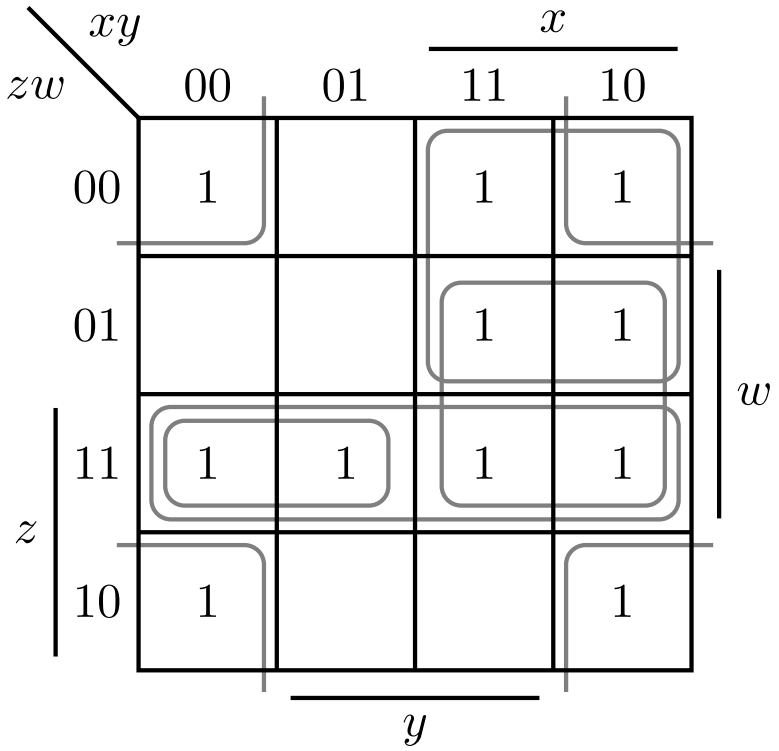
- Combinando le caselle si riduce il numero di letterali in un termine prodotto, diminuendo di conseguenza il costo dell'espressione
- In analogia con quanto sarebbe derivabile dall'interpretazione «spaziale» dei cubi Booleani, su una K-map a 3 variabili:
  - Una casella rappresenta un mintermine con 3 variabili
  - Due caselle adiacenti rappresentano un termine prodotto con 2 variabili
  - Quattro termini adiacenti rappresentano un termine prodotto con 1 variabile
  - Otto termini adiacenti rappresentano la funzione costante 1
- Nel caso generale sono ammissibili combinazioni che includono solo un numero di celle adiacenti pari a una potenza di 2, con raggruppamenti che assumano una forma rettangolare (un quadrato è un caso particolare di rettangolo)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

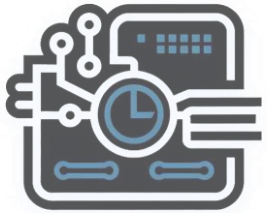
Esempi di raggruppamenti ammissibili







2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



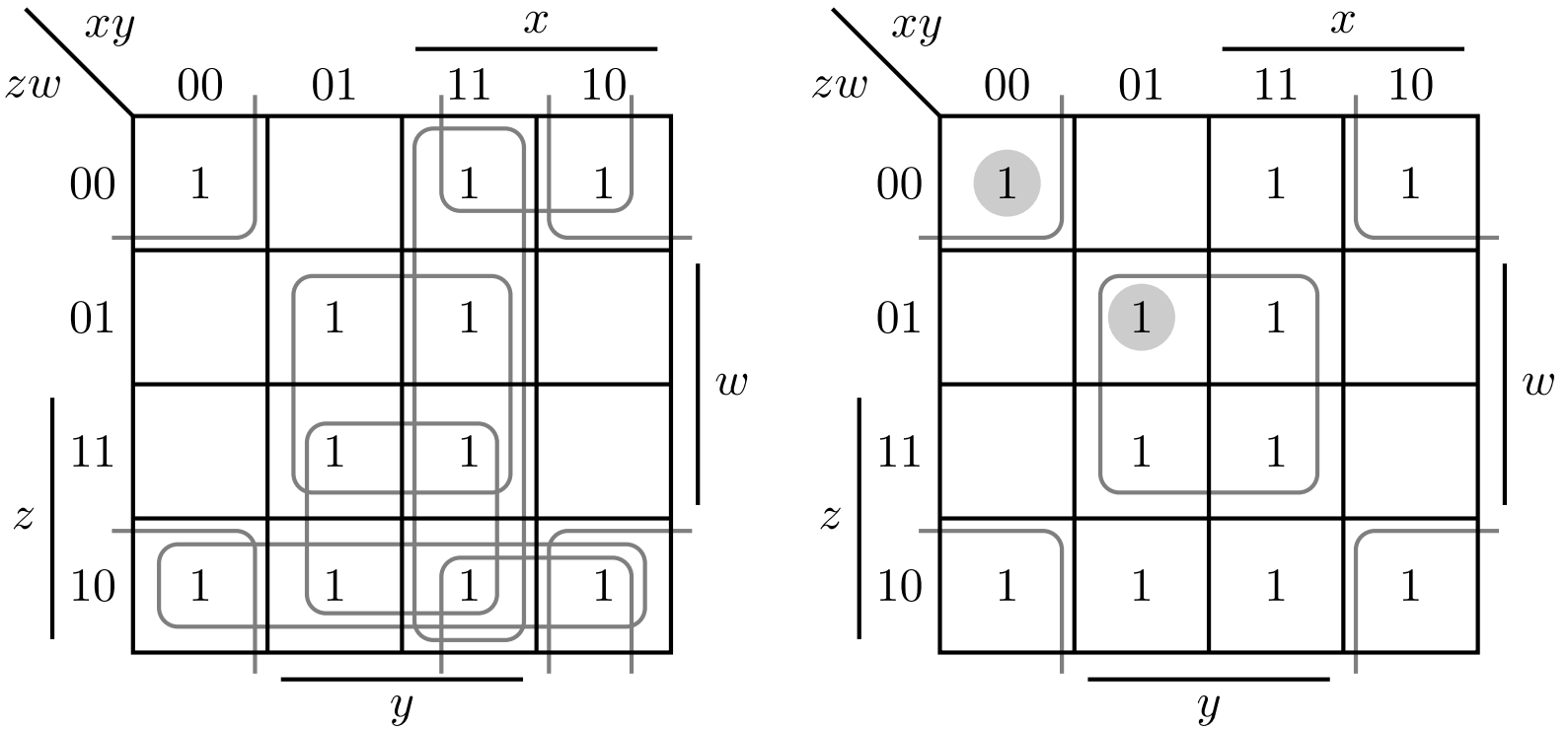
Implicanti, primi ed essenziali

- Date due espressioni Booleane *f* e *g*, *g* è un **implicante** di *f* se *f* vale *1* ogniqualvolta *g* valga *1*
- Si definisce **implicante primo** un implicante che non sia coperto a sua volta da un implicante con meno letterali ovvero nessun suo sottoinsieme proprio è un implicante
- Implicanti non primi possono essere esclusi da una espressione Booleana senza alterarne la natura, poiché ridondanti
- Un implicante primo si dice **essenziale** se è il solo implicante primo che copre uno o più mintermini
- Un insieme di implicanti primi copre tutti i mintermini se, per ogni mintermine della funzione, almeno un implicante primo nell'insieme degli implicanti primi include tale mintermine

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio di implicanti primi ed essenziali

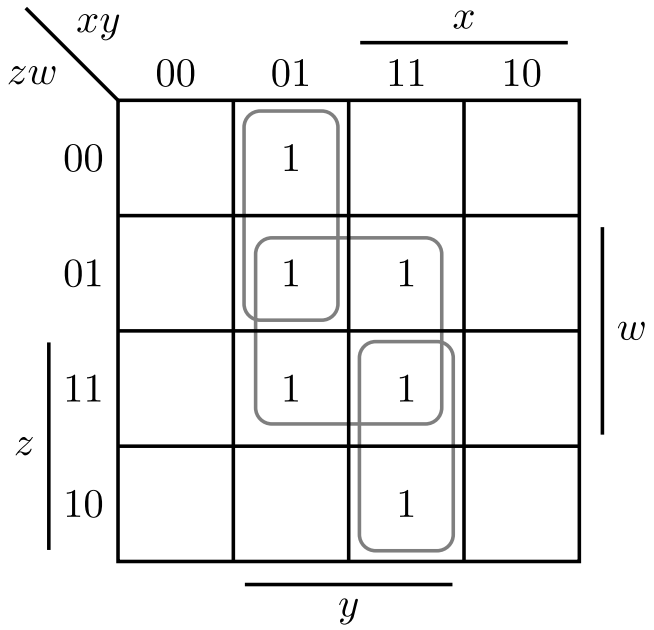
- Due esempi di copertura per una generica funzione in 4 variabili in cui sono elencati tutti gli implicanti primi, a sinistra, e i soli implicanti primi essenziali, a destra, evidenziando i mintermini che li rendono tali



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

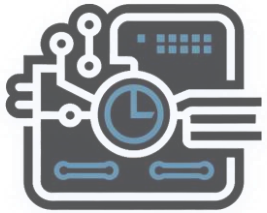
Esempio di minimizzazione con uso di K-map

- Andiamo a identificare i raggruppamenti più ampi possibili, tenendo conto del poter usare più volte la medesima cella
- Si possono identificare tre implicanti primi essenziali, uno di ordine *2* e due di ordine *1*. Sono i termini che andremo a scegliere per la rappresentazione finale della funzione



$$f(x, y, z, w) = yw + \overline{x}y\overline{z} + xyz$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



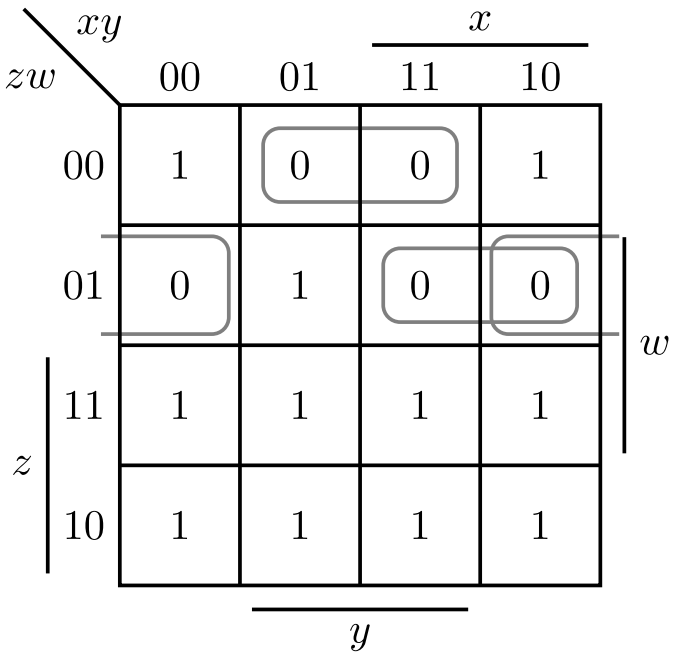
Algoritmo (informale) di minimizzazione

- Trovare tutti gli implicanti primi
- Includere nella soluzione tutti gli implicanti primi essenziali
- Selezionare un insieme a costo minimo di implicanti primi non essenziali per coprire tutti i mintermini non ancora coperti
- Minimizzare il più possibile la sovrapposizione tra implicanti primi
- Assicurarsi che ogni implicante primo scelto includa almeno un mintermine non incluso in nessun altro implicante primo selezionato

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Copertura degli zeri

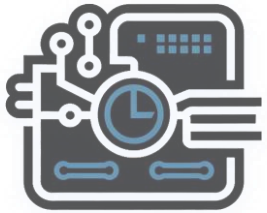
- In maniera duale a quanto introdotto sin ora si sintetizzano gli zero della funzione in una forma OR-AND
  - Fondamento teorico: seconda forma del teorema di Shannon
- Le variabili appaiono con polarità positiva, se assumono valore 0, negativa altrimenti



$$f(x,y,z,w) = (\overline{y}+z+w)(\overline{x}+z+\overline{w})(y+z+\overline{w})$$

$$\overline{f}(x,y,z,w) = y\bar{z}\bar{w} + x\bar{z}w + \bar{y}\bar{z}w$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

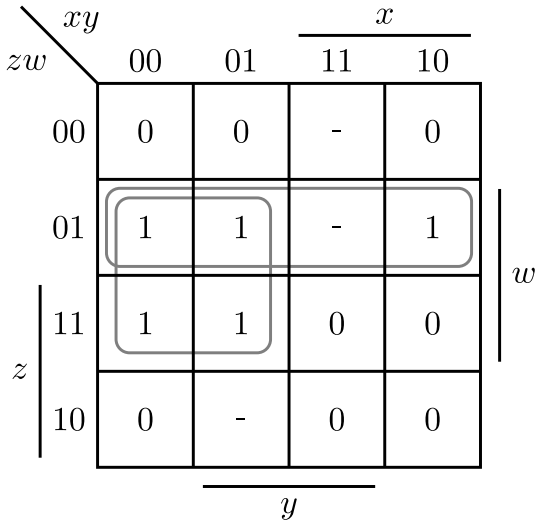


Copertura con «don’t care»

- Le condizioni di **don’t care**, o condizioni di indifferenza, si hanno quando una funzione non è completamente specificata
- Per certe applicazioni la funzione può non essere specificata per alcune combinazioni delle variabili in ingresso e a queste combinazioni corrisponde una condizione di indifferenza
  - Possono rappresentare scenari dove non si sia interessati al comportamento del sistema
- Si indica graficamente tale occorrenza utilizzando il simbolo – oppure una X
- Si possono trattare i *don’t care* come fossero degli zero o degli uno, sulla base di ciò che convenga maggiormente caso per caso
  - È possibile trattare in maniera diversa *don’t care* distinti nella medesima mappa

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

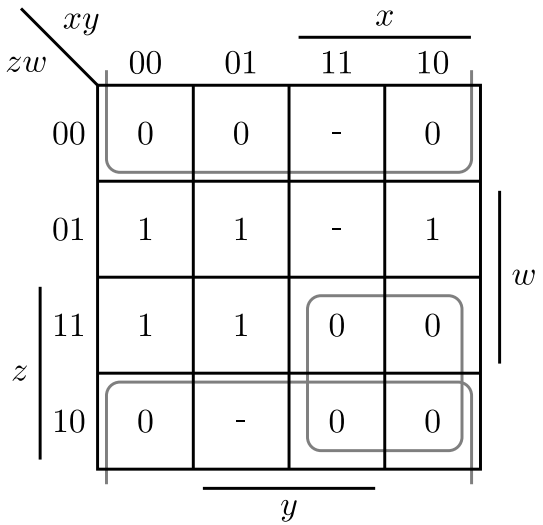
Esempio di copertura con «don’t care»



$$f(x,y,z,w) = \sum_m (1,3,5,7,9) + \sum_d (6,12,13)$$

Trascurando i *don’t care*:

$$f(x,y,z,w) = \bar{x}w + \bar{y}\bar{z}w$$



Sfruttando i *don’t care*:

$$f(x,y,z,w) = \bar{x}w + \bar{z}w$$

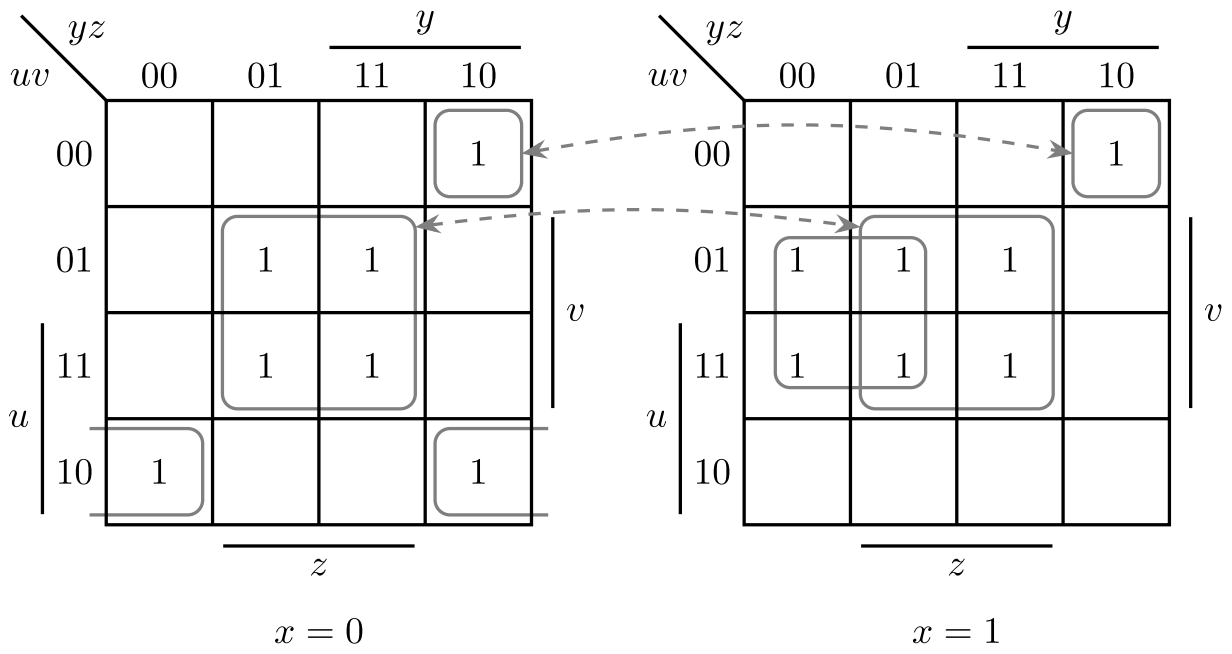
Coprendo gli zeri e sfruttando i *don’t care*

$$f(x,y,z,w) = w(\bar{x} + \bar{z})$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

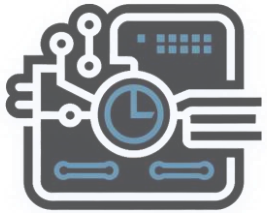
Esempio di K-map con *5* variabili

- Si scompone in due K-map a *4* variabili, logicamente «sovrapposte e allineate» rispetto alla quinta variabile



$$f(x,y,z,u,v) = \sum_m (2,5,7,8,10,13,15,17,19,21,23,24,29,31)$$
$$= zv + x\bar{y}v + y\bar{z}\bar{u}\bar{v} + \bar{x}\bar{z}u\bar{v}$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



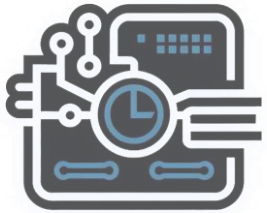
Metodo di Quine-McCluskey (QMC)

- Coincide con la traduzione in algoritmo della minimizzazione basata su K-map
- Si compone di due fasi: «espansione» e «copertura»
- Nella fase di espansione vengono generati tutti gli implicanti primi della funzione da minimizzare
- Nella fase di copertura viene scelto il minor numero di implicanti primi necessari a ottenere una funzione equivalente a quella di partenza
- Entrambe le fasi sfruttano delle tabelle di appoggio per tenere traccia dei progressi fatti dall'algoritmo nella ricerca

- Entrambe le fasi sono illustrate a seguire usando come riferimento la funzione:

$$f(x, y, z) = \sum_{m}(0, 2, 3, 4, 6, 7)$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



QMC – Espansione

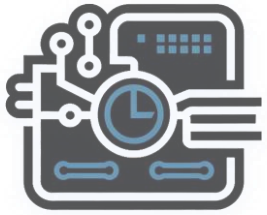
3 variabili

i	x	y	z	p
0	0	0	0	
2	0	1	0	
4	1	0	0	
3	0	1	1	
6	1	1	0	
7	1	1	1	

$$f(x, y, z) = \sum_{m}(0, 2, 3, 4, 6, 7)$$

- Elenca i minterm raggruppando insieme quelli che presentano lo stesso numero di variabili negate
  - Ordina i gruppi per numero di 1 crescenti
- Combina a coppie minterm di gruppi adiacenti per generare implicanti con una variabile in meno rispetto alle variabili iniziali
  - Permette di limitare il numero di coppie da valutare
- Si possono combinare solo minterm che differiscono per una variabile, che appare in forma diretta in uno e negata nell'altro

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



QMC - Espansione

3 variabili

i	x	y	z	p
0	0	0	0	
2	0	1	0	
4	1	0	0	
3	0	1	1	
6	1	1	0	
7	1	1	1	



3 variabili

i	x	y	z	p
0	0	0	0	✓
2	0	1	0	✓
4	1	0	0	✓
3	0	1	1	✓
6	1	1	0	✓
7	1	1	1	✓

- Nell'esempio proposto si combinano tutte le seguenti coppie:
  - (0,2), (0,4), (2,3), (2,6), (4,6), (3,7), (6,7)
- Due minterm combinati vengono marcati come usati, per tenere traccia del fatto che sicuramente non sono implicanti primi
- Considerando la prima coppia (0,2) la combinazione produce un nuovo implicante, con una variabile in meno, descritto come:

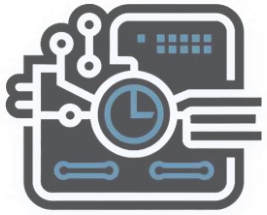
$$m_0 + m_2 = \bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} = \bar{x}\bar{z}(y + \bar{y}) = \bar{x}\bar{z}$$

$$000 + 010 = 0 - 0$$

$$0 + 2 = (0, 2)$$

- Esaminando tutte le coppie, si ottiene la tabella di implicanti in *n-1* variabili da considerare alla prossima iterazione

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



QMC - Espansione

3 variabili

i	x	y	z	p
0	0	0	0	✓
2	0	1	0	✓
4	1	0	0	✓
3	0	1	1	✓
6	1	1	0	✓
7	1	1	1	✓

2 variabili

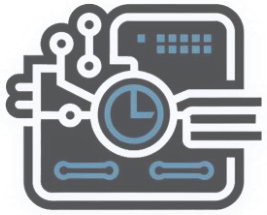
i	x	y	z	p
$0,2$	0	$-$	0	
$0,4$	$-$	0	0	
$2,3$	0	1	$-$	
$2,6$	$-$	1	0	
$4,6$	1	$-$	0	
$3,7$	$-$	1	1	
$6,7$	1	1	$-$	

- Si ripete il medesimo ragionamento visto al passo precedente, tentando di eliminare una ulteriore variabile
  - Obiettivo è ancora generare la tabella di implicanti in *n-1* variabili, rispetto al passo attuale
- In qualsiasi passo dell'espansione, è potenzialmente possibile ottenere il medesimo risultato combinando termini diversi.
  - Basta tenere traccia di una singola istanza di un certo implicante risultante
  - Marcare comunque gli implicanti delle varie alternative come «usati»

$$(0,2)+(4,6)=(0,2,4,6)$$
$$0-0+1-0=--0$$

$$(0,4)+(2,6)=(0,4,2,6)$$
$$-00+-10=--0$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



QMC - Espansione

3 variabili

i	x	y	z	p
0	0	0	0	✓
2	0	1	0	✓
4	1	0	0	✓
3	0	1	1	✓
6	1	1	0	✓
7	1	1	1	✓

2 variabili

i	x	y	z	p
0, 2	0	–	0	✓
0, 4	–	0	0	✓
2, 3	0	1	–	✓
2, 6	–	1	0	✓
4, 6	1	–	0	✓
3, 7	–	1	1	✓
6, 7	1	1	–	✓

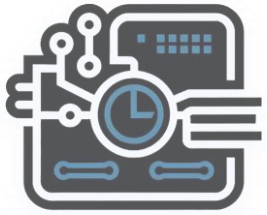
1 variabile

i	x	y	z	p
0, 2, 4, 6	–	–	0	p_0
2, 3, 6, 7	–	1	–	p_1

- Il procedimento termina quando non è possibile combinare ulteriormente implicanti
- Tutti gli implicanti non marcati sono automaticamente degli implicanti primi
- La fase di espansione si può considerare conclusa

$$p_0 = (0, 2, 4, 6) = - - 0 = \overline{z}$$
$$p_1 = (2, 3, 6, 7) = -1- = y$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



QMC - Copertura

- Ha come scopo individuare un insieme di implicanti primi sufficiente a coprire la funzione di partenza
- Serve garantire che ogni minterm della funzione originale sia coperto da (almeno) un implicante
- Serve garantire l'utilizzo del minor numero possibili di implicanti
- Sfrutta una cosiddetta *tabella di copertura*
- Colonne in cui sia presente una singola spunta, rappresentano degli implicanti primi essenziali
- Nel caso non fosse possibile coprire con soli implicanti essenziali, è necessario effettuare una copertura iterativa con gli implicanti rimanenti
- È un problema che ricade nella classe di complessità NP-completo

	m_0	m_2	m_3	m_4	m_6	m_7
p_0	✓	✓		✓	✓	
p_1		✓	✓		✓	✓

$$f(x, y, z) = p_0 + p_1 = \overline{z} + y$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

QMC – Copertura (Es. 2)

	m_1	m_2	m_3	m_6	m_9	m_{10}	m_{11}	m_{12}
p_0								✓
p_1								✓
p_2	✓		✓		✓		✓	
p_3	✓				✓			
p_4		✓	✓			✓	✓	
p_5		✓		✓		✓		

	m_1	m_3	m_9	m_{11}	m_{12}
p_0					✓
p_1					✓
p_2	✓	✓	✓	✓	
p_3	✓		✓		
p_4		✓		✓	

	m_1	m_3	m_{12}
p_0			✓
p_1			✓
p_2	✓	✓	
p_3	✓		
p_4		✓	

	m_1	m_3	m_{12}
p_0			✓
p_2	✓	✓	

- Si selezionano gli implicanti primi essenziali, e si rimuovono le righe corrispondenti dalla tabella (p_5)
  - Criterio di *essenzialità*
- Una colonna può essere eliminata se è interamente coperta da (almeno) un'altra colonna (es: m_9 da m_1)
- Una riga può essere eliminata se è coperta interamente da (almeno) un'altra riga (es: p_3 da p_2)
  - Criteri di *dominanza*
- Gli implicanti primi rimanenti sono necessari alla copertura
- In alcuni casi non è possibile fare considerazioni di questo tipo, e diventa necessario esaminare tutte le possibili combinazioni di implicanti

$$f(x, y, z, w) = p_0 + p_2 + p_5 = xy\overline{z} + \overline{y}w + z\overline{w}$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

QMC – Criterio di dominanza per righe

	m_1	m_2	m_3	m_6	m_9	m_{10}	m_{11}	m_{12}
p_0								✓
p_1								✓
p_2	✓		✓		✓		✓	
p_3	✓				✓			
p_4		✓	✓			✓	✓	
p_5		✓		✓		✓		

	m_1	m_3	m_9	m_{11}	m_{12}
p_0					✓
p_1					✓
p_2	✓	✓	✓	✓	
p_3	✓		✓		
p_4		✓		✓	

	m_1	m_3	m_{12}
p_0			✓
p_1			✓
p_2	✓	✓	
p_3	✓		
p_4		✓	

	m_1	m_3	m_{12}
p_0			✓
p_2	✓	✓	

- Un implicante p_i domina un implicante p_j quando p_i copre almeno tutti i minterm coperti da p_j
- La riga associata a p_j può essere eliminata, conservando solo la riga associata a p_i
- L'eliminazione di una riga può generare dei nuovi implicanti essenziali
- Le righe ad essi associate vengono chiamate righe essenziali secondarie (implicanti primi secondari)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

QMC – Criterio di dominanza per colonne

	m_1	m_2	m_3	m_6	m_9	m_{10}	m_{11}	m_{12}
p_0								✓
p_1								✓
p_2	✓		✓		✓		✓	
p_3	✓				✓			
p_4		✓	✓			✓	✓	
p_5		✓		✓		✓		

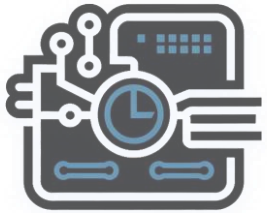
	m_1	m_3	m_9	m_{11}	m_{12}
p_0					✓
p_1					✓
p_2	✓	✓	✓	✓	
p_3	✓		✓		
p_4		✓		✓	

	m_1	m_3	m_{12}
p_0			✓
p_1			✓
p_2	✓	✓	
p_3	✓		
p_4		✓	

	m_1	m_3	m_{12}
p_0			✓
p_2	✓	✓	

- Un minterm m_i domina un minterm m_j quando ogni implicante che copre m_j copre anche m_i
- La colonna associata a m_i può essere eliminata, conservando solo la colonna associata a m_j
  - Prestare attenzione allo scambio al target della cancellazione, che è scambiato rispetto al criterio di dominanza per righe!

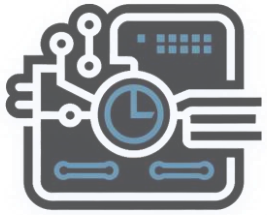
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Metodo di «Espresso»

- Il metodo di Quine-McCluskey presenta dei limiti pratici:
  - Il numero di implicanti primi cresce esponenzialmente con il numero di ingressi, proporzionalmente a $3^n/n$
  - Individuare una copertura minima rientra nella classe di problemi NP-completi
- *Espresso*, introdotto da Brayton e Sangiovanni nel 1984, sacrifica la minimalità della soluzione a fronte di una procedura più rapida:
  - Non genera tutti gli implicanti primi durante la fase di espansione
  - Sceglie oculatamente un sottoinsieme di implicanti primi adatti a coprire l'onset
  - Replica in maniera più simile il modo con cui un essere umano esaminerebbe una K-map

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

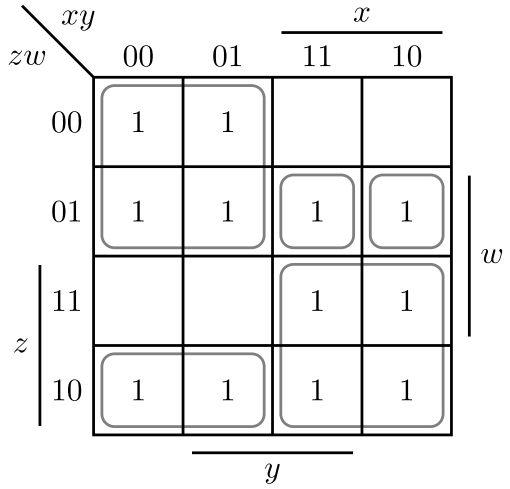


Espresso - Ciclo Expand / Irredundant Cover / Reduce

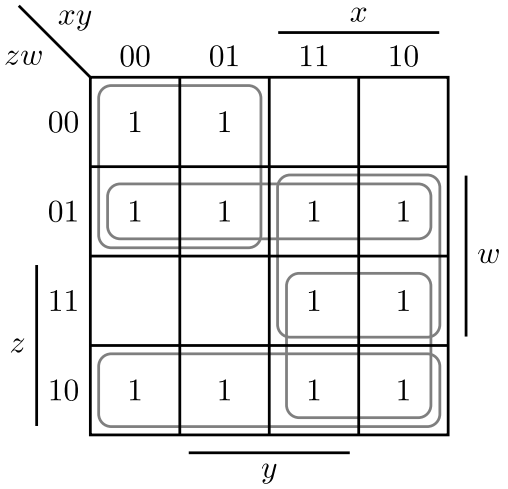
- Inizia da una copertura iniziale non ridondante ma non necessariamente minima
- Passo EXPAND
  - Espande euristicamente gli implicanti fino alla loro dimensione massima
- Passo IRREDUNDANT COVER
  - Estrae una copertura non ridondante dall'insieme degli implicanti, in analogia con la fase di copertura di QMC
- Passo REDUCE
  - Poiché potenzialmente potrebbe esistere una soluzione migliore rispetto a quella trovata, riduce/contrare ogni implicante alla dimensione che ancora copre l'onset
- Ripete il ciclo E-IC-R per tentare altre soluzioni fino a convergenza
- Concettualmente, può essere visto come una serie di movimenti nello spazio delle soluzioni dove mediante EXPAND/REDUCE ci si sposta da possibili minimi locali

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

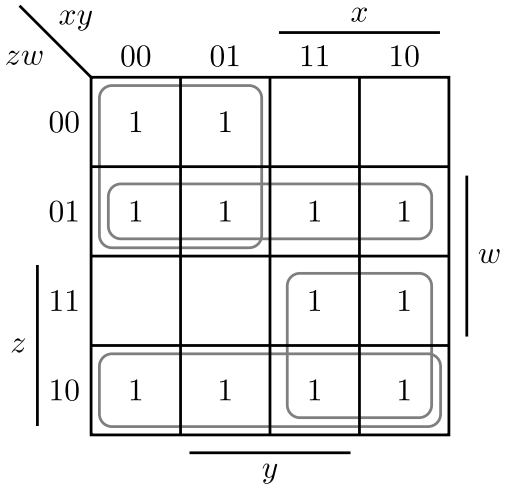
Espresso - Ciclo Expand / Irredundant Cover / Reduce



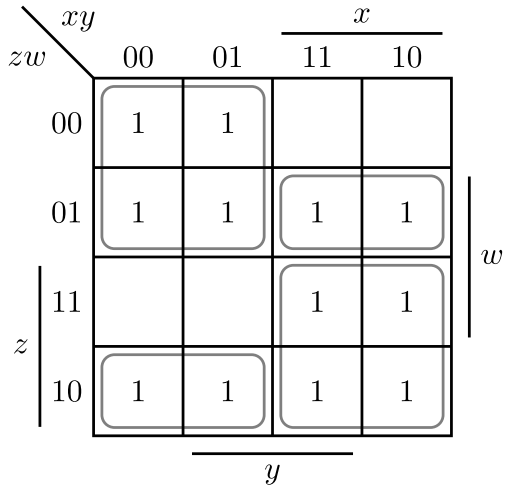
ASSUME



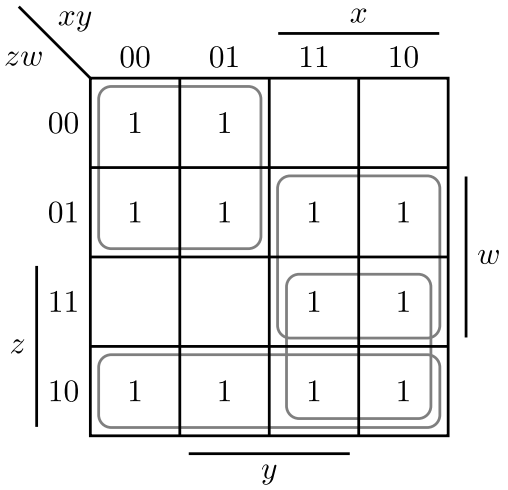
EXPAND



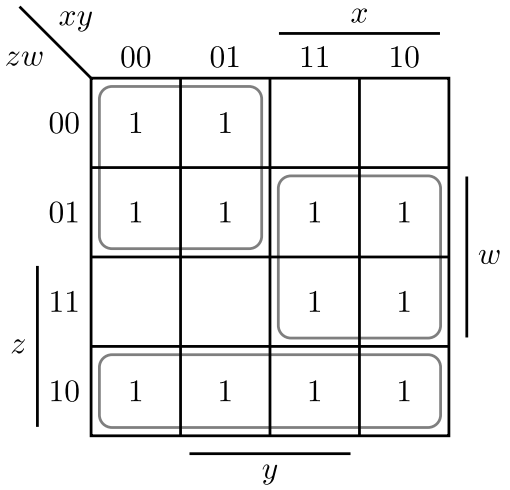
IRREDUNDANT



REDUCE

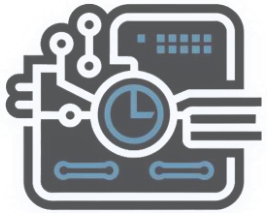


EXPAND



IRREDUNDANT

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



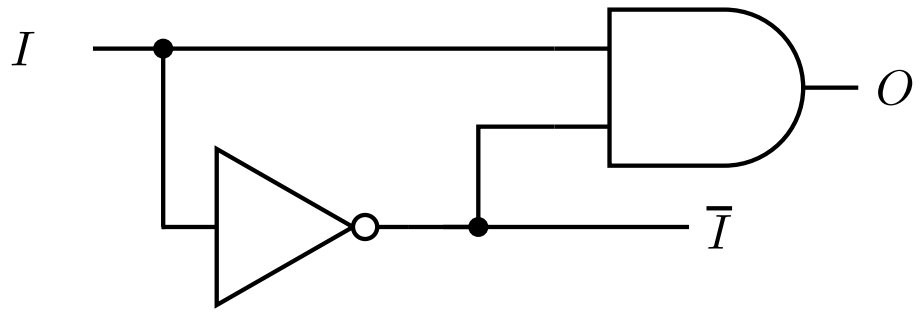
Comportamento nel tempo delle reti combinatorie

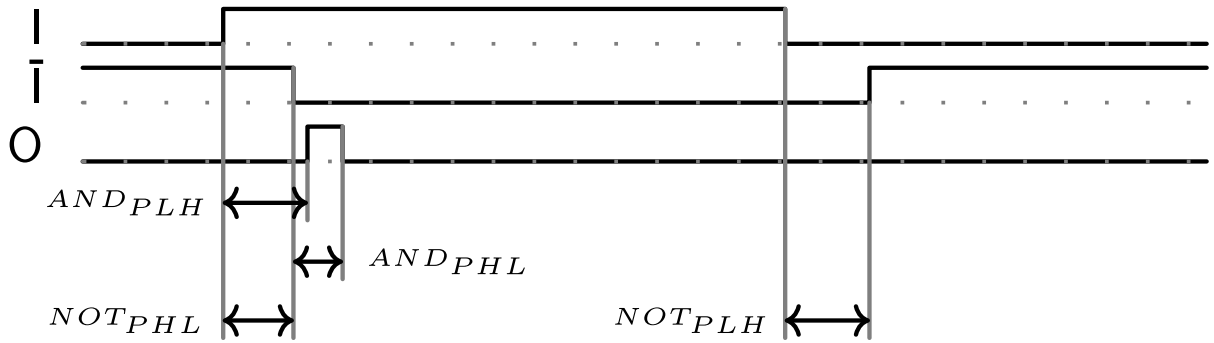
- Una porta logica presenta un ritardo tra il cambiamento del valore d'ingresso e quello di uscita
- Serve tenere conto dei tempi di transizione (propagazione) da livello basso ad alto (PLH) e viceversa (PHL). Questo tipo di ritardo prende il nome di ritardi di trasporto
- Un ulteriore livello di realismo è ottenibile andando a considerare uno scenario in cui le transizioni non avvengono in tempo nullo, ma avvengono in un tempo finito e seguono un andamento lineare. In questo caso si parla di ritardi inerziali
- Per la discussione a seguire, terremo conto dei soli ritardi di propagazione

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Alee (Hazards)

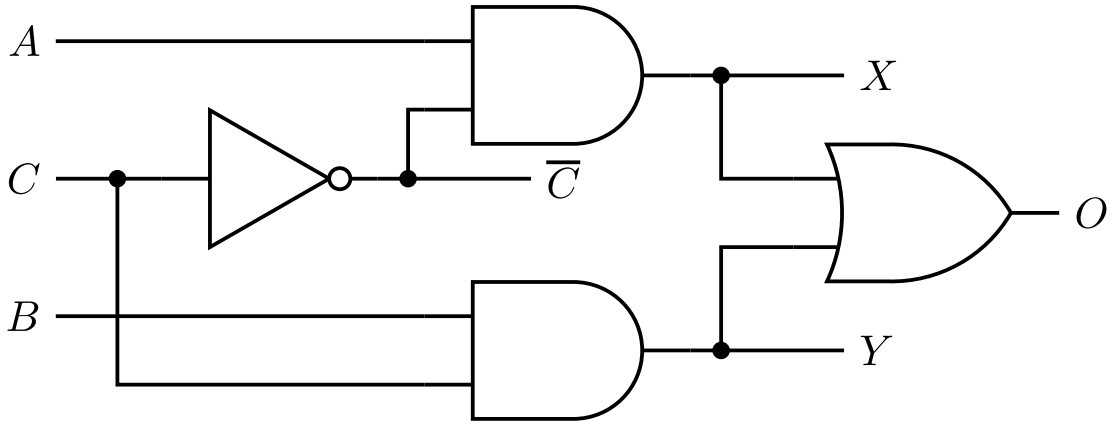
- Si tratta di comportamenti impulsivi imputabili ai ritardi di trasporto
- A seconda della tecnologia utilizzata, possono essere mascherate dai ritardi inerziali
- Le alee possono essere eliminate, o almeno mascherate, sfruttando strumenti algebrici
- Le alee che nascono in presenza di percorsi di ritardo asimmetrici, dovuti alla presenza di invertitori o altre porte, sono chiamate *alee statiche*

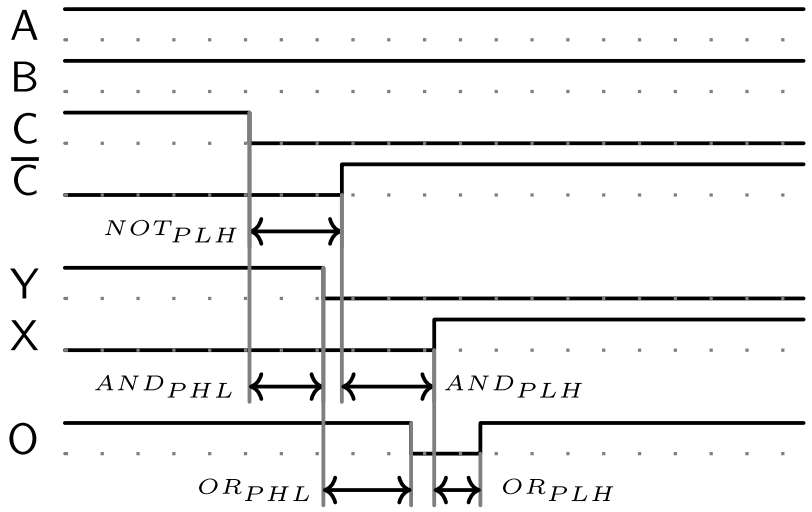




2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Eliminazione delle alee statiche

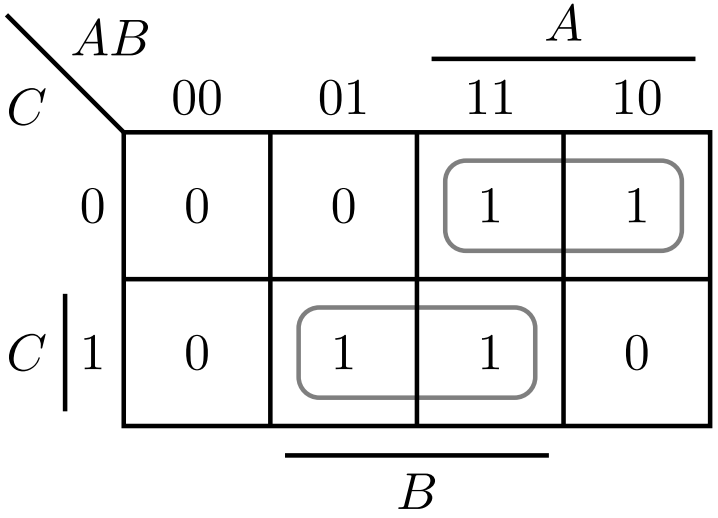




- Il circuito dovrebbe presentare uscita sempre al livello logico alto, poiché codifica un selettore tra il segnale A e il segnale B, modulato da C
- A causa del ritardo sulla porta NOT, esiste un momento in cui entrambe le uscite dei gate AND passano a zero
- Come conseguenza, si verifica la presenza di un'alea statica sull'uscita, in tale frangente, che la porta temporaneamente al livello logico basso

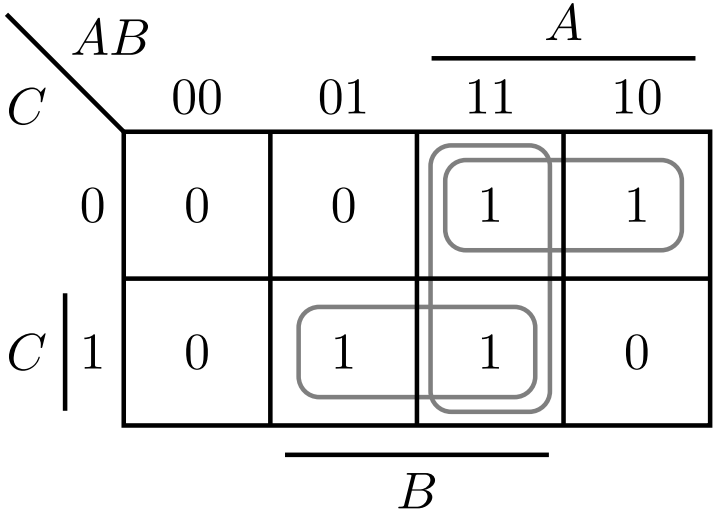
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Eliminazione delle alee statiche



- L'alea si sviluppa nella transizione 111 > 110, al passare del segnale C dal valore 1 al valore 0

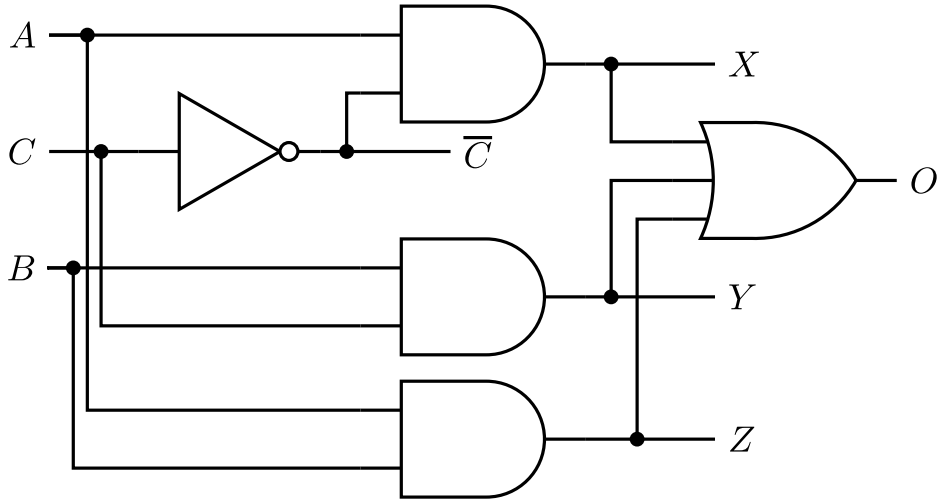
Teorema: Una rete combinatoria a due livelli, realizzata come SOP, è libera da alee se è libera da alee per tutte le transizioni 1 > 1

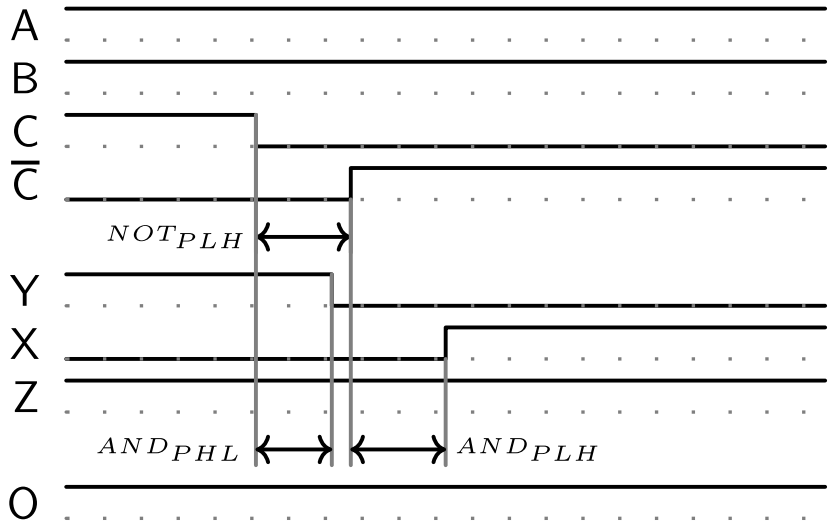


- In una rete combinatoria AND-OR a due livelli si può manifestare un'alea in presenza di una coppia di 1 adiacenti, non coperti dal medesimo implicante
- Per eliminare le alee statiche, si procede in modo sistematico aggiungendo implicanti non essenziali in tutte le situazioni che possono produrre alee

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

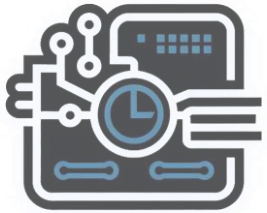
Eliminazione delle alee statiche





- Il circuito precedente viene modificato introducendo un termine prodotto aggiuntivo, e cambiando il numero di termini della somma
- La sintesi non è più minima, ma il nuovo termine maschera l'alea sulla transizione problematica

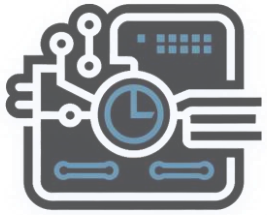
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Argomenti di approfondimento individuale

- A1) Teorema di espansione di Shannon
- A2) Cubi Booleani

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Teorema di espansione di Shannon – Prima forma

A1

- Una generica funzione Booleana può essere scomposta come segue:

$$f(X_1, X_2, X_3, \ldots, X_n) = \overline{X_1} \cdot f(0, X_2, X_3, \ldots, X_n) + X_1 \cdot f(1, X_2, X_3, \ldots, X_n)$$

- Il ragionamento è iterabile su tutte le variabili:

$$f(X_1, X_2, X_3, \ldots, X_n) = \overline{X_1} \cdot \overline{X_2} \cdot f(0, 0, X_3, \ldots, X_n) + \overline{X_1} \cdot X_2 \cdot f(0, 1, X_3, \ldots, X_n) + X_1 \cdot \overline{X_2} \cdot f(1, 0, X_3, \ldots, X_n) + X_1 \cdot X_2 \cdot f(1, 1, X_3, \ldots, X_n)$$

- Il passo finale è caratterizzato dall'aver "esposto" tutti i termini prodotto, ognuno moltiplicato per un valore costante ovvero il valore assunto dalla funzione per ogni combinazione di variabili.

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Teorema di espansione di Shannon – Prima forma

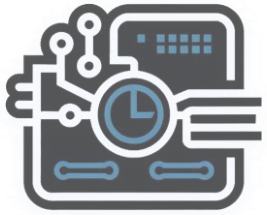
X_1	X_2	X_3	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$
\begin{aligned}
f(X_1, X_2, X_3) = {} & \overline{X_1} \cdot \overline{X_2} \cdot \overline{X_3} \cdot f(0,0,0) + \\
& \overline{X_1} \cdot \overline{X_2} \cdot X_3 \cdot f(0,0,1) + \\
& \overline{X_1} \cdot X_2 \cdot \overline{X_3} \cdot f(0,1,0) + \\
& \overline{X_1} \cdot X_2 \cdot X_3 \cdot f(0,1,1) + \\
& X_1 \cdot \overline{X_2} \cdot \overline{X_3} \cdot f(1,0,0) + \\
& X_1 \cdot \overline{X_2} \cdot X_3 \cdot f(1,0,1) + \\
& X_1 \cdot X_2 \cdot \overline{X_3} \cdot f(1,1,0) + \\
& X_1 \cdot X_2 \cdot X_3 \cdot f(1,1,1)
\end{aligned}
$$

$$
\begin{aligned}
f(X_1, X_2, X_3) = {} & \overline{X_1} \cdot \overline{X_2} \cdot \overline{X_3} \cdot 0 + \\
& \overline{X_1} \cdot \overline{X_2} \cdot X_3 \cdot 1 + \\
& \overline{X_1} \cdot X_2 \cdot \overline{X_3} \cdot 0 + \\
& \overline{X_1} \cdot X_2 \cdot X_3 \cdot 0 + \\
& X_1 \cdot \overline{X_2} \cdot \overline{X_3} \cdot 0 + \\
& X_1 \cdot \overline{X_2} \cdot X_3 \cdot 1 + \\
& X_1 \cdot X_2 \cdot \overline{X_3} \cdot 1 + \\
& X_1 \cdot X_2 \cdot X_3 \cdot 0
\end{aligned}
$$

$$
\begin{aligned}
f(X_1, X_2, X_3) = {} & \overline{X_1} \cdot \overline{X_2} \cdot X_3 + \\
& X_1 \cdot \overline{X_2} \cdot X_3 + \\
& X_1 \cdot X_2 \cdot \overline{X_3}
\end{aligned}
$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Teorema di espansione di Shannon – Seconda forma

- Una generica funzione Booleana può essere scomposta come segue:

$$f(X_1, X_2, X_3, \ldots, X_n) = (X_1 + f(0, X_2, X_3, \ldots, X_n)) \cdot (\overline{X_1} + f(1, X_2, X_3, \ldots, X_n))$$

- Il ragionamento è iterabile su tutte le variabili:

$$\begin{aligned} f(X_1, X_2, X_3, \ldots, X_n) = {} & (X_1 + X_2 + f(0, 0, X_3, \ldots, X_n)) \cdot \\ & (X_1 + \overline{X_2} + f(0, 1, X_3, \ldots, X_n)) \cdot \\ & (\overline{X_1} + X_2 + f(1, 0, X_3, \ldots, X_n)) \cdot \\ & (\overline{X_1} + \overline{X_2} + f(1, 1, X_3, \ldots, X_n)) \end{aligned}$$

- Il passo finale è caratterizzato dall'aver "esposto" tutti i termini somma, ognuno addizionato ad un valore costante ovvero il valore assunto dalla funzione per ogni combinazione di variabili.

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Teorema di espansione di Shannon – Seconda forma

X_1	X_2	X_3	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$
\begin{aligned}
f(X_1,X_2,X_3) = {} & (X_1+X_2+X_3+f(0,0,0))\cdot \\
& (X_1+X_2+\overline{X_3}+f(0,0,1))\cdot \\
& (X_1+\overline{X_2}+X_3+f(0,1,0))\cdot \\
& (X_1+\overline{X_2}+\overline{X_3}+f(0,1,1))\cdot \\
& (\overline{X_1}+X_2+X_3+f(1,0,0))\cdot \\
& (\overline{X_1}+X_2+\overline{X_3}+f(1,0,1))\cdot \\
& (\overline{X_1}+\overline{X_2}+X_3+f(1,1,0))\cdot \\
& (\overline{X_1}+\overline{X_2}+\overline{X_3}+f(1,1,1))
\end{aligned}
$$

$$
\begin{aligned}
f(X_1,X_2,X_3) = {} & (X_1+X_2+X_3+0)\cdot \\
& (X_1+X_2+\overline{X_3}+1)\cdot \\
& (X_1+\overline{X_2}+X_3+0)\cdot \\
& (X_1+\overline{X_2}+\overline{X_3}+0)\cdot \\
& (\overline{X_1}+X_2+X_3+0)\cdot \\
& (\overline{X_1}+X_2+\overline{X_3}+1)\cdot \\
& (\overline{X_1}+\overline{X_2}+X_3+1)\cdot \\
& (\overline{X_1}+\overline{X_2}+\overline{X_3}+0)
\end{aligned}
$$

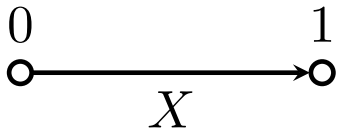
$$
\begin{aligned}
f(X_1,X_2,X_3) = {} & (X_1+X_2+X_3)\cdot \\
& (X_1+\overline{X_2}+X_3)\cdot \\
& (X_1+\overline{X_2}+\overline{X_3})\cdot \\
& (\overline{X_1}+X_2+X_3)\cdot \\
& (\overline{X_1}+\overline{X_2}+\overline{X_3})
\end{aligned}
$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

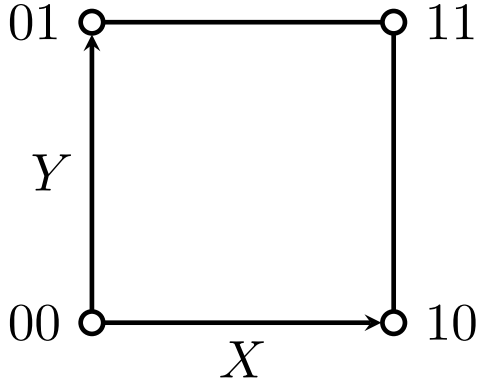
Cubi Booleani

A2

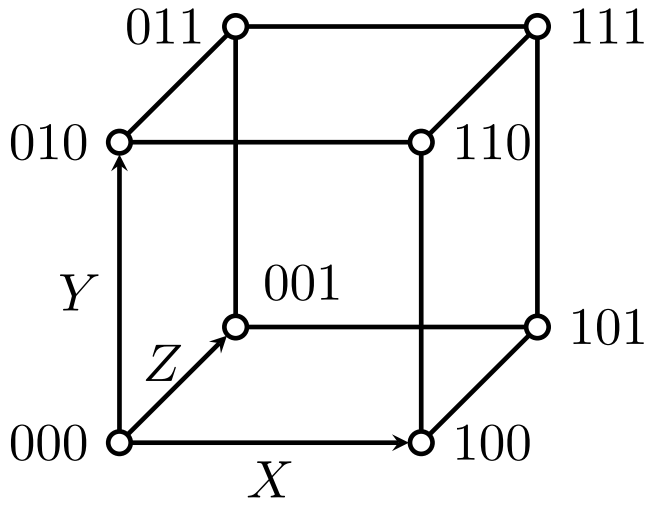
- Sono una tecnica visuale che permette di identificare più facilmente l'applicabilità del teorema di unificazione
- Un cubo ha tante dimensioni quante sono le variabili considerate



1−cube



2−cube

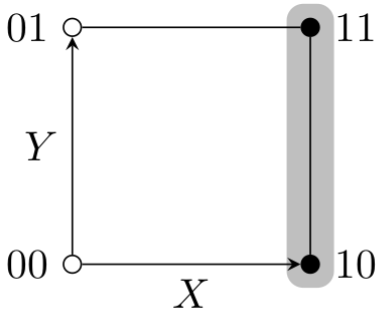


3−cube

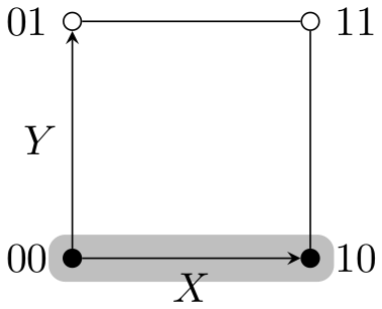
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio: 2-cubo

- Consideriamo due funzioni *f* e *g* in cui rappresentiamo l'on-set come vertici pieni e l'off-set come vertici vuoti



f



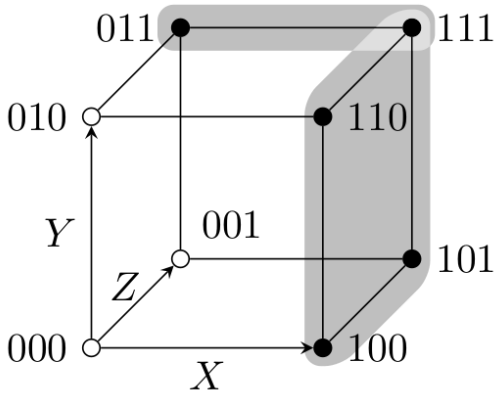
g

- I lati evidenziati in grigio rappresentano dei cubi con *n-1* dimensioni, la cui espressione prodotto associata contiene quindi *n-1* variabili
- Per la funzione *f*, nel sotto-cubo, la variabile *X* è sempre vera, mentre *Y* cambia
- Per la funzione *g*, nel sotto-cubo, la variabile *X* cambia, mentre *Y* è sempre falsa

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

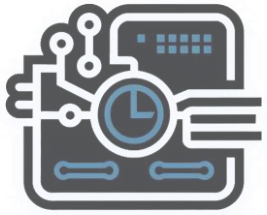
Esempio: 3-cubo

- Consideriamo una funzioni *f* in cui rappresentiamo l'on-set come vertici pieni e l'off-set come vertici vuoti



- Per lo spigolo superiore, si applicano le stesse considerazioni viste in precedenza
- La faccia destra, evidenziata, rappresenta un'espressione dove la variabile *X* è sempre vera, e le altre due cambiano. Identifica quindi un termine prodotto di una singola variabile, che coincide esattamente con il letterale *X*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Considerazioni sugli n-cubi

- In un 3-cubo, si ha che:
  - uno *0*-cubo (un solo vertice) è un termine con tre letterali
  - un *1*-cubo (uno spigolo con due vertici) è un termine con due letterali
  - un *2*-cubo (un piano con quattro vertici) è un termine con un letterale
  - un *3*-cubo (un cubo con otto nodi) è la costante *1*

- Generalizzando, si può affermare che un *m*-cubo sottoinsieme di un *n*-cubo, con $m<n$, è un termine di $n-m$ letterali

- Rimane vero che non sia ancora particolarmente comodo operare direttamente sui cubi, specialmente se dobbiamo spingerci in più dimensioni

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini